CARELINK FOLLOW-UP · DAVID MWELWA · SOFTWARE ENGINEER

Help clinic staff find patients who missed an appointment, even when the internet is down

The problem as I understood it

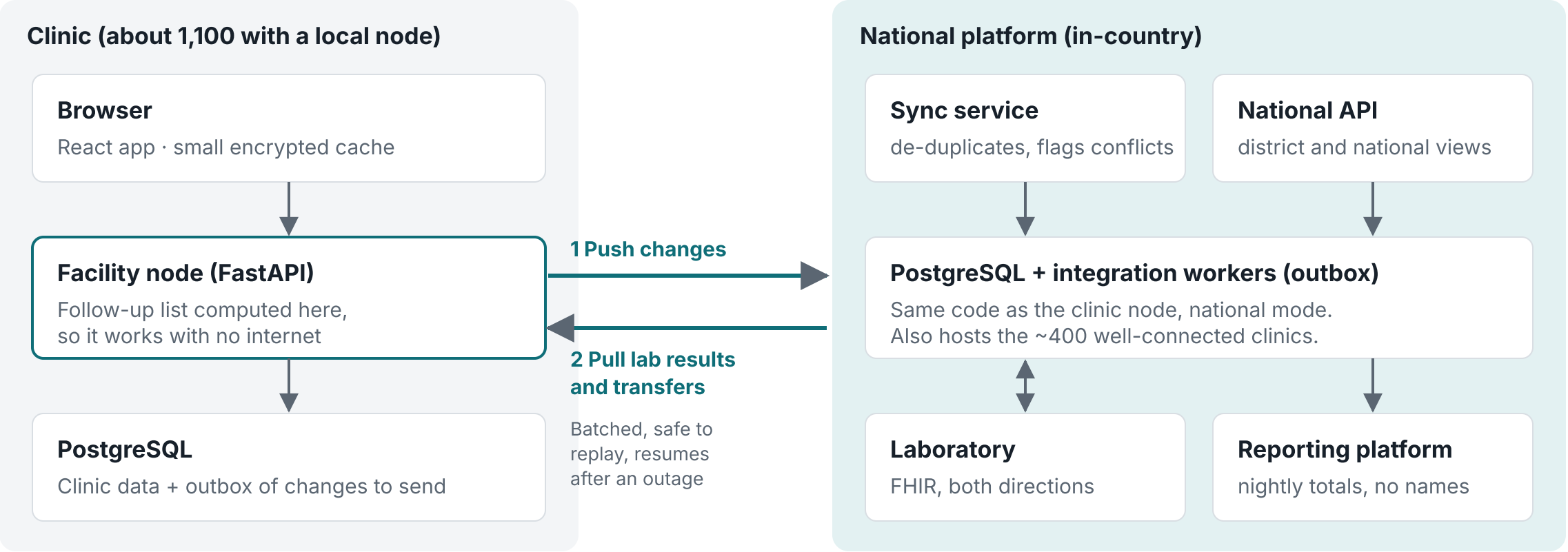
- 1,500 clinics; 300 offline for hours or days
- Old 4 GB computers, keyboard-only use, lists over 10,000 patients
- Legally protected health data
- Releases go out while clinics are open

Assumptions where the brief was unclear

- Clinics already have a local CareLink database (Section G mentions one)
- A visit *on* the appointment day counts as attending
- "More than 7 days" is strict: day 7 is not yet included, day 8 is
- Missed = 8 to 30 days; overdue = over 30, matching the sample data
- "Today" means today in Lusaka

SECTION A · ARCHITECTURE

Each clinic works against its own copy; the national server is reached only by sync



Three decisions I am confident about, and one I am not

CHOSE	REJECTED, AND WHY
Modular monolith	Microservices: too much to run for a team of nine in nine months; a clinic node must be one unit
PostgreSQL outbox	Kafka: I know it well, but this load is modest and it is one more system to secure in-country
Human review of clinical conflicts	Last-write-wins: clinic clocks drift, and silently losing a clinician's edit is unsafe

Least confident: a node at about 1,100 clinics

It rests on my assumption that clinics already run a local CareLink database with reliable power and someone to restart it.

What would change my mind: evidence that many sites lack that hardware or support. Then I would move to a browser-first offline app with a smaller cache, and accept that two PCs in one clinic only share work after sync.

SECTION B · WHAT I BUILT (LIVE DEMO)

One endpoint and one screen, filtering done in PostgreSQL

API: `GET /api/follow-up`

- Finds each patient's latest visit with one indexed query, then filters, sorts and pages in the database
- Clinicians see only their facility; district officers read across facilities and cannot write (`403`)
- One error shape with a correlation ID; no names or phone numbers in logs
- `POST …/contacted` accepts an `Idempotency-Key` , the same idea as Section D

Screen: follow-up list

- Status shown as words and an icon: Due soon, Missed, Overdue
- Loading, empty and error states with Retry, tested by stopping the API
- Fully keyboard-operable table; filters have labels; errors are announced
- All requests go through a typed service layer

Stubbed: **fill in: "nothing" or what and why**

SECTION B · EVIDENCE

Measured, not asserted

MEASUREMENT	BEFORE	AFTER	WHAT CHANGED
API page 1, 100k patients / 400k visits	__ ms	__ ms	Composite index on visits, read with `EXPLAIN ANALYZE`
10,000-row list, render time (4× CPU slowdown)	__ ms	__ ms	Row virtualisation + server paging
10,000-row list, DOM nodes	__	__	Only visible rows exist

Tests focus on the rule's edges

Day 7 vs day 8; a visit on the appointment day; a later visit with no new appointment; same-day visits; each status; `401` / `403`; the error shape.

Why those

A wrong boundary means a nurse phones someone who came, or never phones someone who didn't.

Commands: `make test`

SECTION C · CODE REVIEW

What I would block before merge

Back-end service (C1), which I refactored

- **Critical** SQL built from user input: a search box can read or delete any patient
- **Critical** Errors swallowed, no transaction: the nurse sees "saved", the visit is gone
- **Critical** Admin password in the code; national IDs in the logs

Chosen because silent data loss and a breach are harder to detect and undo than a broken screen.

Front-end component (C2)

- **Critical** Wrong parameter and response shape: the list is empty, so "nobody needs follow-up"
- **Critical** No error handling: "loading" forever, no retry
- **Critical** "Mark contacted" fails silently: patients phoned twice or not at all

Also: polling every 3 s from 3,000 users ≈ 1,000 large requests a second.

SECTION D · OFFLINE AND DUPLICATES, IN PLAIN TERMS

Sending something twice must be harmless, and nobody's work is silently thrown away

Every entry gets a receipt number

Duplicates

It is stamped when the nurse presses Save. If the same entry arrives twice, the server recognises the number and keeps one.

The clinic keeps working

Outages

Work is saved locally with a visible count of what is waiting. When the line returns it sends in small batches, picks up where it stopped, and never re-sends what was confirmed.

A person decides conflicts

Clashing edits

If two people changed the same detail, both versions are shown side by side and the clinician chooses. Visits are only ever added, so they never clash.

Existing duplicate patients are never deleted: a person confirms each pair, and the merge can be undone.

NEXT

With another week, I would

- Build the browser outbox and a sync endpoint, proving Section D in code
- Replace static tokens with Keycloak (OIDC)
- Load-test at 3,000 concurrent users
- Add contract tests between the screen and the API
- Test with a screen reader (NVDA) on an old 4 GB machine

Also in the submission

- **F:** stop fixes on production via a fast hotfix lane; CI every change must pass
- **G:** phased rollout: data from week 2, all facilities by week 6

Thank you. Questions?